



Adaptive user interface based on accessibility context

Lamia Zouhaier¹ · Yousra BenDalyHlaoui¹ · Leila Ben Ayed¹

Received: 10 March 2022 / Revised: 4 December 2022 / Accepted: 6 January 2023 /

Published online: 9 March 2023

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

The substantial involvement of Adaptive User Interfaces (AUI) in providing adaptive and accessible interactive systems has created the need to establish a multimodal framework based on scalable adaptation rules. This paper presents an Adaptive User Interface to Accessibility Context (AUIAC) framework that provides a generic adaptation approach according to the model-driven engineering. It is based on a sequential and layered transformation from platform independent model (PIM) to platform specific model (PSM). It supports different reifications and transitions using adaptive transformation rules specified for each disability and modality. We illustrate the application of some rules on a sample user interface for the case of blind people. Then, we present some usability evaluation results from an empirical study.

Keywords Adaptive user interfaces · Accessibility · Metamodel transformation · Model-driven engineering

1 Introduction

Human-Computer Interaction (HCI) is a huge domain. It includes more than fourteen branches specified by a large number of methods, techniques, patterns, and standards for conception, design, implementation and evaluation of interactive systems [10]. This has allowed the rise of modern User Interfaces(UI) [5, 10, 17], often called Intelligent UI or [25] Adaptive UI, as a result of new emerging technologies such as Ontologies, Artificial Intelligence, Cloud Computing and IoT. Modern UIs are equipped with a fast, efficient, optimized and valuable adaptivity to any context change in contrast to classical UIs. Indeed,

Yousra BenDalyHlaoui and Leila Ben Ayed are contributed equally to this work.

✉ Lamia Zouhaier
zouhaier.lamia@yahoo.fr

Yousra BenDalyHlaoui
yousra.hlaoui@fst.utm.tn

Leila Ben Ayed
leila.benayed@ensi-uma.tn

¹ LIPSIC Lab, Faculty of Science of Tunis, Tunis El Manar University, Tunis, Tunisia

the development of Intelligent Model-Based UI [2] has enabled significant scalability since it addresses diverse user models, diverse platforms and heterogeneous ambient environment which launches Grand Challenges for adapting user interfaces to context change [23] impacting human life quality [19].

Adapting a user interface (UI) for accessibility purposes is completely different from adapting the UI for any other context and purpose. The definition of UI adaptation as a whole is the customization of the UI structure, content, or behavior based on the users' context [8]. In contrast, UI adaptation for accessibility purposes is the customization of UI structure and content to fit the user's disability and other contextual data with respect to World Health Organization's [36] recommendation and certain accessibility norms as WCAG2.0 [11, 15] and the norm ISO 9241-171 [14].

People with disabilities (PWD)s [36] experience barriers when they interact with classical UIs [3, 4]. These latter are developed with obsolete methods, with very slow responses, unable to be sensitive to context changes and incapable to be adapted to user' needs and abilities. For that reason, Intelligent UI, equipped with technological innovations is considered as adaptive [5] since it adapts automatically to any change of context by adjusting itself to what is requested at run-time. Hence, we consider that adaptive interfaces are effective solutions for accessibility that require genericity to consider a large type of PWDs [17] and to be independent from implementation and platforms.

In the other side, the heterogeneity of disability profiles implies the multiplicity of accessibility aspects that must be considered, for instance, user ability, user preferences, user needs, user mobility and user educational level [4]. Moreover, computing platforms present diverse characteristics with high performances used for the interaction with the UIs. This can lead to several solutions to match users disabilities. In addition, the ubiquitous environment comprises different aspects like light, noise, social environments, location, proximity, time. All these aspects raise new challenges of the adaptation process to enhance interaction between PWDs and platforms. Thus, we have defined accessibility context using information describing the user disabilities, the used computing platform and the physical environment of the computing platform at the user situation. Thus, we have developed an ontology specifying the *Accessibility Context* via the *Disability Profile*, the *User Identity*, the *User Situation*, the *Technical Context* and the *Environment Context* [4]. The choice of ontology is mainly due to its relevance to knowledge management for a better semantic representation of the accessibility context and its suitability for IoT-based applications [32].

In this paper, we present the AUIAC system supporting a generic approach for the adaptation of UIs to the accessibility context. This approach provides a solution for different users with disabilities. The genericity of the approach represents a key point as our ultimate goal is performing an automatic adaptation of the UI by considering a large number of disabilities based on the Model-Driven Engineering [21]. In fact, genericity generates a sustainable adaptation process independent of technology-specific platforms. The genericity of the process allows us to focus on the adaptation of different aspects and types of UI builders independently of the different characteristics of the technologies supporting the non-adapted UIs. Thus, we improve the ability to reason about the domain of our focus which is the adaptation of UIs to the accessibility context and the corresponding solution.

Currently, AUIAC has major limitations:

- It is tested only for graphical user interfaces.
- It does not consider persons with intellectual, cognitive and physical impairments.

Therefore, AUIAC system is implemented by a series of meta-model transformations between concrete and abstract models of the non-adapted UI to adapt. Its is composed of four cooperative modules:

1. *UI Reverse Engineering Module (UIRE module)*: serves to load an abstract model (Platform Independent Model) from the non-adapted UI.
2. *Accessibility Knowledge Management Module (AccKM module)*: supplies knowledge about the captured context to the AccAP module. For that reason, we have chosen the ontology paradigm to model our accessibility context.
3. *Accessibility Adaptation ProcessModule (AccAP module)*: represents the kernel of the approach where different meta-model transformations are performed taking as inputs the non-adapted UI model resulted from UIRE module and the accessibility context model from AccKM module.
4. *Cross-Platform-Adaptation Process Module (CrossPAP module)*: transforms the adapted PIM model to a specific UI model based on the executed platform.

The modules *AccAP* and *CrossPAP* are complementary since they include three consecutive metamodel transformations: the transformation *T1-PIM2PIM*, the transformation *T2-RefinedPIM2PIM* and the transformation *T3-PIM2PSM*. The different transformations are specified through a set of transformation rules between non-adapted UI meta-model and adapted UI meta-model considering the accessibility context ontology as a parameter.

The paper will be organized in the following order: In Section 2, we discuss some related literature. In the next section, we provide an in-depth description of the AUIAC system. Then, the proposed metamodel transformations are described in Section 4. In Section 5, we present some results of the empirical usability evaluation study. Finally, Section 6 presents some concluding remarks and future work.

2 Related work

Researchers and practitioners in the field of HCI are working to make available accessibility-related solutions. Different type of classification were proposed in order to compare different proposed approaches of user interfaces adaptation for accessibility basically according to adaptive and adaptable systems. Firmenich et al. [8] discusses that adaptive systems could be classified according to the development approach (Model-driven Engineering, window managers, widget toolkits) the architectures (close adaptive, open adaptive), techniques (tailored UI, Rule-based UI, Context-based UI), methods (model-based or framework-based). Another classification is related to user/system implication at different stages of the adaptation. AVANTI [34] is approached as the first systems to take advantage of the fields of rehabilitation design and automatic system adaptation in terms of content, navigation and presentation for people with disabilities. It categorizes users according to different stereotypes and, as a result, it presents optional content and selects appropriate information from the alternatives.

Authors in [7] propose an accessible graphical UI which supports well-designed default parameters for chart generation. Besides, VERITAS framework [29] allows the inclusion of accessibility requirements at the outset of software design. It gives simulation of how PWDs interact with UI. Also, in [16], a framework proposes the design of an accessibility-inclusive blindness-friendly UI for performing common activities on a mobile devices. This framework re-organizes/re-generates the UI components into a simplified blindness-friendly UI

based on user profile and contextual recommendations. In addition, in [22], authors propose strategy to promote web accessibility by the use of software awareness in legacy systems. To provide a personalized accessibility, they have used self-adaptation strategies to provide customized accessibility. Moreover, Global Public Inclusive Infrastructure (GPII) [35] aims to provide PWDs with accessible public services automatically. The authors in [16] have proposed a Universal Accessibility Framework (UAF) that generates a simplified UI from mobile application artifacts automatically based on user information model, context model and device usage logs. Also, SUPPLE [9] proposes an automatic personalizing of UIs suitable for people with motor and vision impairments. They have adapted the UIs to person's abilities, devices, preferences, and tasks. EGOKI [1] is an automatic generator of accessible UIs allowing people with disabilities to access supportive ubiquitous services. EGOKI follows a model-based approach and selects suitable interaction resources and modalities depending on users capabilities. Finally, MyUI [24] promises the adaptivity as a solution for the accessibility of user interfaces. It provides individualized user interfaces which are accessible to a broad range of users. It composes different design patterns for specific interaction situations and contextual information related to users, environments and platforms. The adaptation process is based on a predefined set of accessibility patterns. Therefore, the adaptation process is limited as it is based on

These approaches (AVANTI, EGOKI, SUPPLE, MyUI, GPII) are oriented context-based UI adaptation but not generic as it is based on a finite set of individualization patterns which could not cover all aspects and features of UIs and accessibility context to adapt UIs. However they only tackle a very restricted range of UI type and specific impairments. In contrast to our contribution, we consider almost kind of UIs(graphical,vocal, multimodal). So, our approach is not restricted to a set of individualization patterns nor to a specific kind of UI. The major limitation of these automated approaches is that they have applied the customization of UIs based on specific prototypes and patterns. However, in our approach we adopt a model-based method considering tailored UI, using context-based adaptation using Rule-based adaptation. Our approach aims to adapt any kind of UI (desktop, web, mobile) to any kind of user disability (sensory, cognitive, motor) for a large number of execution platforms and using different suitable modalities(vocal, graphical multimodal).

In our approach, we propose a generic adaptation process based on the given UI models representing all its features and concepts and the accessibility context defining user disability profile, environment profile and platform-technical profile. Hence, the individualization of the given UIs do not depend of a restricted set of adaptation patterns but it depends of the UI components and the accessibility context.

To resume, our system is based on the following contribution: the development of a model-based approach addressing adaptation of UIs to accessibility context. As we are based on models, we ensure the genericity of the approach independently of the diversity of technologies and platforms. Therefore, we focus only on the adaptation considering all UI characteristics and accessibility context concepts: we ensure the knowledge conceptualization in the UI accessibility context aware domain. Besides, we are not limited to a specific kind of UIs nor to a finite set of individualization patterns to perform our adaptation process. In fact, we consider given UIs based on different modalities which will be adapted to any kind of user disability, any type of platform surrounded by any kind of environment. Finally, we build an ontology describing the accessibility context which will be used in the adaptation process ensuring the genericity of the approach and we transform any non-adapted UI model into any different specific and running platform, based on meta-model transformation.

3 The AUIAC system

The AUIAC framework is characterized by an evolutionary, modeling-based approach to support the adaptivity of user interfaces to the accessibility context of the users with disabilities. Accessibility is recognized as a standard for quality. So, a UI is considered accessible if it is suitable for people with disabilities. They usually face barriers when using UIs that are not adapted to their abilities. These barriers are mainly determined by the natural impairment of each user. In order to accommodate a variety of disabilities such as hearing and visual impairments (low vision and blindness), the framework requires that a generic solution be built to enable usability. Furthermore, instead of building an adaptation process for each user interface model, we develop a single adaptation process based on the user interface meta-model and is automated with metamodel transformation. As represented in Fig. 1, we provide a framework to automatically extract a non adapted UI model from an already existing UI and, at the same time, extract ontological instances from the accessibility context, infer knowledge, and then adapt the UI model based on the extracted knowledge from the accessibility context. For the automation of our approach, we need to develop an IoT environment to update ontological instances and a machine learning environment to infer some important user interface features useful for the abstraction of UI model using the reverse engineering. Since the user with disability could be mobile all the time, his ontological instance of the relative context requires to be kept fresh in real time. Furthermore, in order to adapt his/her current user interfaces, we need to infinitely infer the appropriate knowledge from the user's ontological context instance. This systematic inference is based on SWRL rules. The latter is used to minimize parameters and eliminate ambiguities. The *Adaptive UI to Accessibility Context: AUIAC* system adapts user interfaces to a given accessibility context. AUIAC system has several properties:

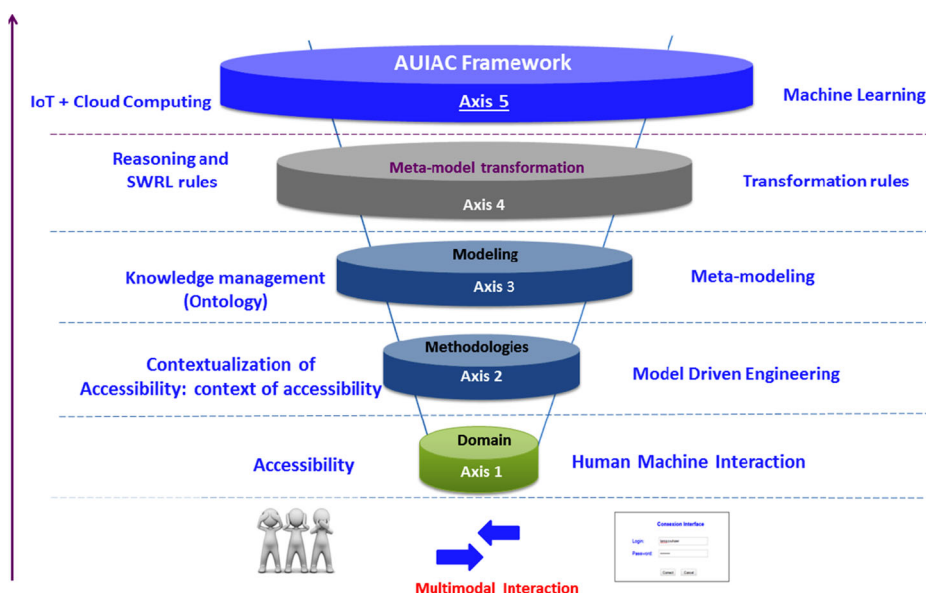


Fig. 1 The approach behind AUIAC system

- The *genericity*: As AUIAC system is applied for a big range of disabilities and platforms (web UI, mobile UI, desktop UI), it has to be generic. Hence, the metamodelling transformation is considered as the kernel of all adaptation modules of the AUIAC system.
- The *adaptivity*: Adaptivity is the basis of the AUIAC system's adaptation process. It provides automatic procedures to adapt the user interface to the accessibility context throughout transformation rules, machine learning algorithms and semantic reasoning on accessibility context. Thus, the system represents the only actor in the AUIAC system. Furthermore, the user with disability is assisted by a chat-bot integrated in AUIAC system to accomplish the interaction with the adapted UI.
- The *intelligence*: AUIAC system is an intelligent system as it integrates machine learning algorithms at different modules. We have implemented an ML-based reverse engineering algorithm to predict the non-adapted PIM model from the current user interface. Moreover, we have based on the Internet of Things platform (IoT platform) for the collection of knowledge about the accessibility context. In addition, we are working on machine learning-driven transformation rules to automate the adaptation process. Therefore, the final adapted UI behaves smartly behind any change in the accessibility context.
- The *context-awareness*: In the AUIAC system, we have developed a context-awareness system, in which we sense, capture, collect, manage and reason information about accessibility context based on two cooperative platforms: the IoT platform (1) and the knowledge management platform (2). This property leads to build a pervasive AUIAC system.
- The *multimodality*: AUIAC system is based on multimodality. It matches each disability with its suitable interaction modality. We opted for this property because it represents a low-cost solution to provide an accessible user interface for each disability profile. Furthermore, it ensures comfort, freedom, and well-being for users with disabilities.
- The *cross-platform*: Devices are multiple and their use is excessive which creates a need to afford a solution that is not limited to a specific platform. Therefore, we proceed with a platform-independent model (PIM model), to free the UI from any relative platform details. AUIAC system has implemented platform-based transformations rules to match the executed platform.
- The *UX-friendliness*: Accessibility of UI aims to provide an accessible friendly-UX (User eXperience). UX refers to the quality of the user with disability experience faced in any interaction. It has to describe the overall experience felt by the user with disability interacting with the adapted UI deployed in any platform. Therefore, to guarantee friendship with UX, the resulted adapted UI has to be usable and ergonomic. We have worked on UI properties and style.
- The *extensibility*: AUIAC system is not limited to a specific kind of disability or platform. It targets all of them. Meta-modelling ensures the re-usability for whatever kind of platform. In addition, the accessibility context is modeled through an ontology which makes it flexible to add new properties or objects in the previous model.
- The *scalability*: AUIAC supports the ability to adapt to the change in magnitude of the demand, especially its ability to maintain its functionality and performance in case of high demand.

AUIAC approach is defined through different cooperative modules. Each module has its goal and is represented by its specific models. Figure 2 represents these modules:

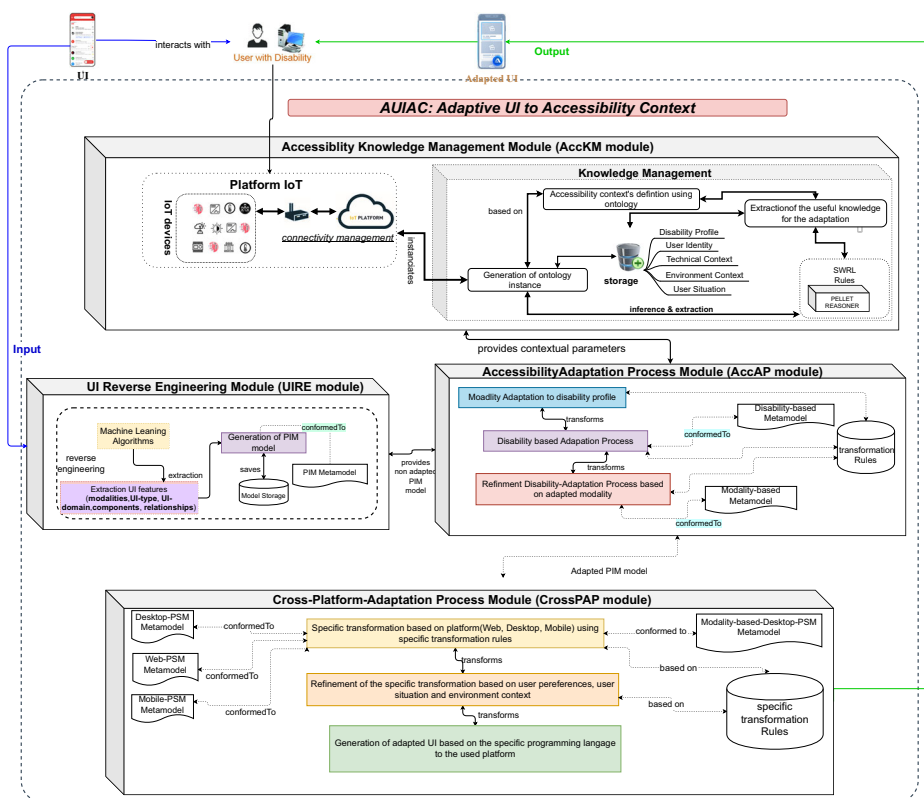


Fig. 2 AUIAC system: an adaptive UI to accessibility context

1. **Accessibility Knowledge Management Module (AccKM module):** The AccKM module defines the process for accessibility context knowledge management. The accessibility context is described through ontologies. The accessibility context ontology represents knowledge about almost accessibility concepts such as the user disability profile (cognitive, physical and sensory disabilities), the user profile (name, age, education, preferences, etc.), the technical context (the used platform, its characteristics, the supported assistance technologies, the supported browsers, its computation capabilities, its interaction devices, the network capacities, etc.) and the environment context (the user location, the interaction time, the physical and social environments) and the user situations describing his/her mobility. The AccKM module communicates with the AccAP module by providing accessibility contextual parameters which serve as inputs for transformation rule engine. The AccKM module is composed of two layers: the first one is dedicated to the IoT Platform in order to collect data from sensors and manage it. It is connected to the cloud through an IoT gateway. AccKM module connects remote platforms to the user with disability applications. The second layer is dedicated to extract knowledge from collected data and then to generate an ontology instance. This instance is used by the reasoning engine that will extract only pertinent accessibility contextual parameters.

2. **UI Reverse Engineering Module (UIRE module):** This module is related to the abstraction of the interactive system's UIs. It concerns the abstract modelling of the non-adapted-UI using a Reverse Engineering technique. In fact, we built a PIM model from the non-adapted UI. The PIM-model which is independent of any platform details, represents a set of abstract and structural components. The PIM model is conformed to the adapted UI meta-model. It has to describe almost of UI types. It includes information about the employed *modalities (input, output)*, the *UI-type* which defines if it is a mobile, a desktop, or a web UI. It includes, also, information about the *UI-domain* which could be gaming, e-learning, healthcare, e-tourism, etc. This kind of information is very useful since we have proposed a series of supervised and non supervised Machine learning algorithms to extract and predict these UI features and to generate the the PIM model. Each generated model will be archived and stored.
3. **Accessibility Adaptation Process Module (AccAP module):** AccAP module is the core of AUIAC system. It concerns the adaptation process of the non-adapted-UI according to the user disability profile and to the modality. It includes three adaptation phases. The first one is the *Modality-based Adaptation*. It adapts the input and the output modalities to the user disability profile to suit user capabilities. The second one is the *Disability-based Adaptation*. It ensures the adaptation of the non-adapted PIM model to the user disability profile. The third one is the *Refinement of Disability based-Adaptation* based on the adapted modalities. It concerns the refinement of the already adapted PIM model taking into account the already adapted interaction modalities. Each phase of the AccAP module is automated by a parametrized meta-model transformation between two meta-models: a source metamodel and a destination metamodel. Each meta-model transformation is based on transformation rules.
4. **Cross-Platform Adaptation Process Module (CrossPAP module):** CrossPAP represents the specific adaptation process to the platform. It provides an executed final adapted UI. It includes three phases: The first one transforms the *adapted UI-PIM* provided from the *AccAP module* to a specific model *Modality-based PSM* depending on the modalities and the platform (web, mobile, desktop). The second one defines a refinement of the *Modality-based PSM* based on user preferences, user situation and environment context. The third one represents the coding step by which we implements the *Refined Modality-based PSM* using a programming language which is the language of the initial non-adapted UI.

The AccAP and CrossPAP modules in our AUIAC system is defined through a number of meta-model transformations.

4 Meta-model transformation within the AUIAC system

AUIAC system is based on an adaptive approach based on meta-modelling and meta-model transformation. The definition of a meta-model transformation as represented in Fig. 3 is given by the association of the quadruplet:

**{Source meta-model (A), Destination meta-model (B),
Accessibility Context Ontology, Transformation rules}.**

- Source meta-model (A): to be adapted, the input UI model is conformed to the source metamodel.
- Destination meta-model (B): the output resulted UI model which was adapted, has to be conformed to the destination metamodel.

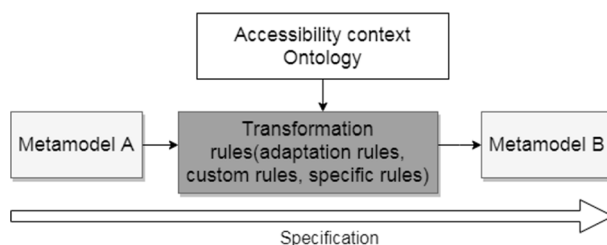


Fig. 3 Definition of AUIAC meta-model transformation

- **Accessibility Context Ontology:** represents knowledge about the accessibility context which is collected from the IoT platform and managed into ontology instances. We consider, at each knowledge layer, specific parameters about disability profile, the technical context and the environment.
- **Transformation rules:** defines how transforming a component to an adapted component. Each transformation has some specific rules which could be adaptation rules, custom rules or specific rules [4].

In order to maximize AUIAC effectiveness, all transformations are knowledge-driven as shown in Fig. 4. We aim to enforce the efficiency of decision-making ability of AUIAC system modules.

The basic strategy of AUIAC system is to improve adaptation service delivery. That is why, it advocates a knowledge management (KM) system since recent researches have shown that KM improves novelty and agility in capturing, sharing, applying, and creating knowledge.

Therefore, we have centralized transformation rules in Knowledge Base (TKB) of each meta-model transformation layer. In addition, each level specifies a knowledge about the transformation features.

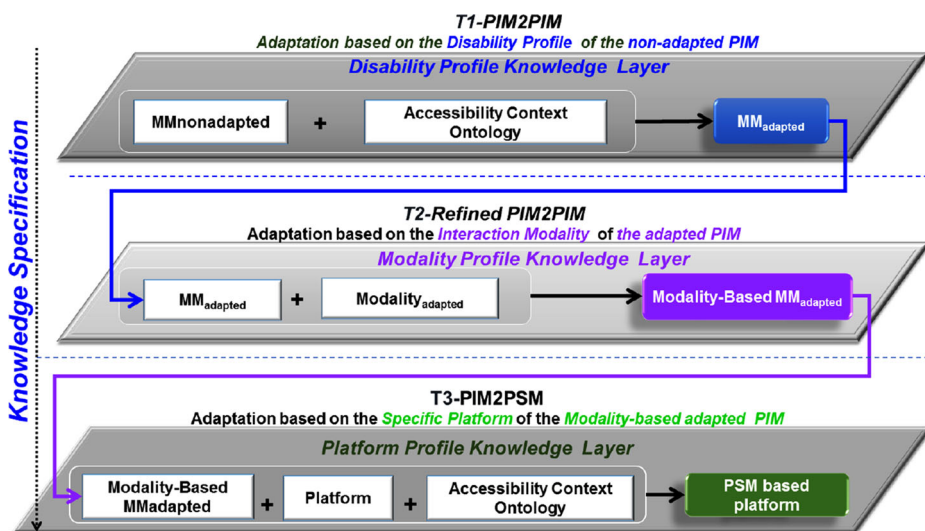


Fig. 4 Knowledge specification of metamodel transformations

The knowledge specification in AUIAC system is composed of three layers:

- **Disability Profile Knowledge Management Layer (Disability-KM layer):** In this layer, we retrieve knowledge about the behavior of the UI model based on the knowledge of the disability profile. This layer is built for accumulation and storage in order to learn and improve understanding of the module AccAP process.
- **Modality Knowledge Management Layer (Modality-KM layer):** this layer retrieves knowledge about the adaptation process based on the interaction modality of the adapted PIM model.
- **Platform Knowledge Management Layer (Platform-KM layer):** this layer retrieves knowledge about the specific adaptation process to the platform. Through the meta-model transformation, we capture and use knowledge about the Modality-based adapted PIM in order to enhance transformation rules.

Therefore, three kind of meta-model transformation are developed in our approach as shown in Fig. 4:

- **T1: Transformation PIM2PIM:** The first transformation defines the adaptation process. It allows the transformation of a non-adapted UI model (non-adapted-PIM) to an adapted UI model (adapted-PIM) by considering only the disability profile values. In fact, *T1 adapts the modalities employed, the structure (UI model Layout, number of elements per container and the element's type) of the non-adapted UI model using user disability profile values.*
- **T2: Refined Transformation PIM2PIM:** The passage from the adaptation process to the specific transformation must imperatively go through an intermediate step that we called a *refined transformation PIM2PIM*. This transformation generates a *refined adapted UI model* which depends on the adapted modality. It generates a novel structure of the UI model with more refined components. *T2 adapts the structure of the adapted UI model resulted from T1 considering the adapted modality.*
- **T3: Transformation PIM2PSM:** It consists on transforming the *adapted UI model* resulted from T2 to an adapted Specific Platform Model (adapted-PSM) depending on the execution platform (web, desktop, mobile). *T3 adapts the style (colour, font, background-colour, text-size, width, height, volume-level, etc) of the refined adapted UI model resulted from T2.* We consider different contextual parameters such as the technical context, the environmental context, user preferences and user situation.

As shown in Fig. 5, the number of transformation is increased using the *divide and rule* paradigm. In addition, the multiplicity of the transformations allows the separation of concerns in the adaptation in order to generate resulted models with more details. The generated new UI instances are specific by the triplet of **disability, modality platform**. The strategy adopted in the AUIAC system helps in reducing the complexity of the adaptation process algorithms. In addition, the genericity of the approach is preserved through the different meta-model transformation which makes it easy in the future, to generate several other specifications of transformations.

4.1 Transformation PIM2PIM (T1)

The first transformation in the AccAP module is a *PIM2PIM* transformation, as it transforms a non-adapted Platform Independent Model (*non-adapted PIM*) of a given UI

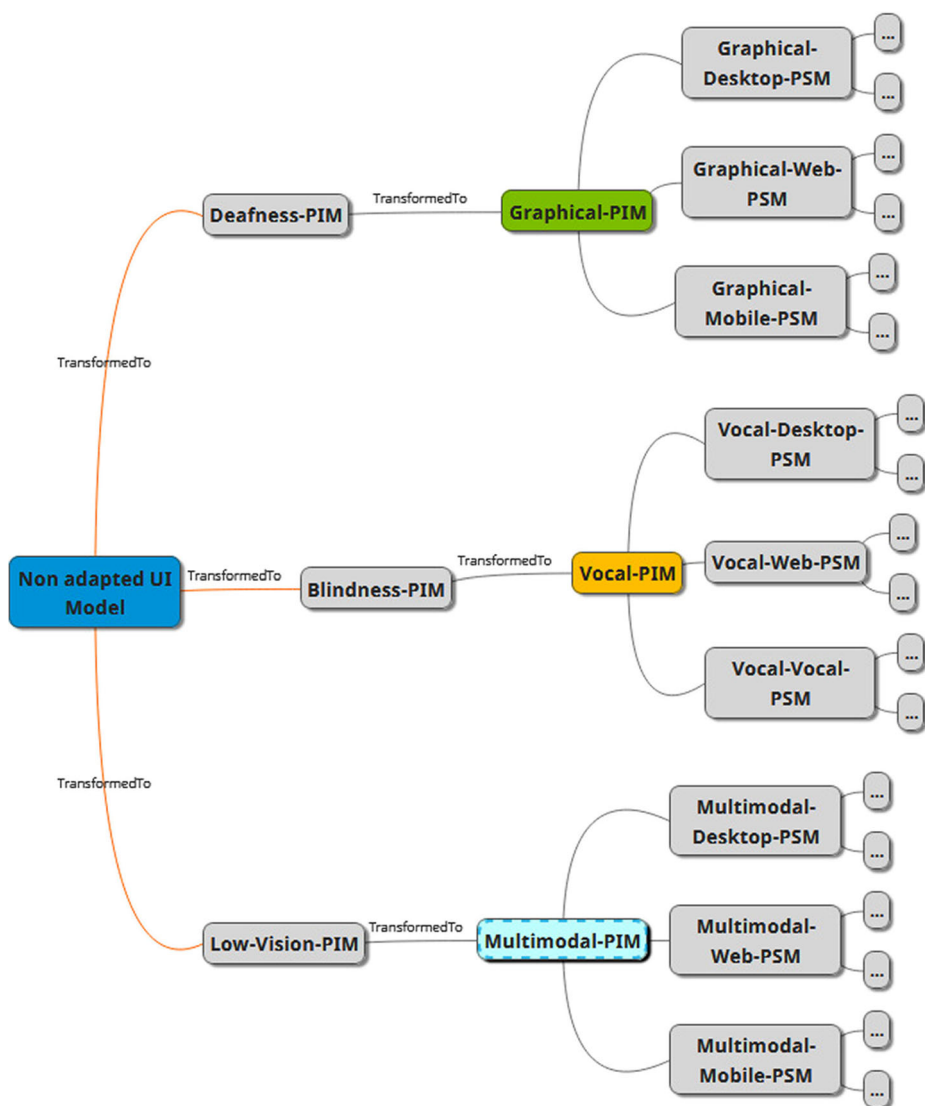


Fig. 5 AUIAC Metamodel Transformations Mind map on the basis of disability, modality, platform

to another *adapted PIM*. It defines the Disability Profile Knowledge Layer since the transformation is based on the knowledge of only the user disability profile.

PIM2PIM transformation includes two kinds of transformations: the adaptation of the modality based on the disability knowledge (*transformation of UI-Model-Modality*) then the adaptation of UI components based on the interaction modalities knowledge (*transformation of UI-model components using the new modality*) [37].

4.1.1 Transformation of UI-model-modality

It consists of adapting the interaction modality of the given UI to the disability profile. It takes as input the accessibility context ontology instance and the UI non-adapted model modality. It provides as output an adapted modality. In fact, it transforms the non-adapted Input and Output modality of the given UI to modalities more suitable to user disability.

In the case of the modality mapping, these rules match a modality Mod of the non-adapted UI meta-model (PIM source meta-model) to its correspondent modality Mod' of the adapted UI meta-model (PIM target meta-model) using the disability profile values depicted from the ontology accessibility context instance $Instance(OO)$. A modality mapping rule is defined as a function m_{mod} . To transform the UI modalities, the function (1) is defined as follows:

$$\begin{aligned} m_{mod} : MM_{nonadapted} \times OO &\rightarrow MM_{adapted} \\ (Mod_{UI_{nonadapted}}, I_{OO}) &\mapsto Mod'_{UI_{adapted}} \end{aligned} \quad (1)$$

$MM_{nonadapted}$: is the source non adapted meta-model

OO is the accessibility context ontology

$MM_{adapted}$: is the target adapted meta-model

$Mod_{UI_{nonadapted}}$: is modality of the UI non adapted model (non adapted PIM) conformed to the source meta-model $MM_{nonadapted}$

I_{OO} is the instance of the accessibility context ontology specifying the user with disability profile.

$Mod'_{UI_{adapted}}$: is the modality of the adapted UI model (adapted PIM) conformed to the target meta-model $MM_{adapted}$

In this case, m is implemented by the Algorithm 1. The complexity is of the order of $O(n)$, where n is the number of classes of the meta model representing the non adapted UI. This algorithm transforms a non-adapted meta-model into an adapted one. It is implemented by kermeta [20] which is a meta-model transformation language that ensures the transformation of models according to the function of conformity between the model and its meta-model.

4.1.2 Transformation of UI-model components using the new modality

After adapting the modality based on the knowledge of the user disability profile, we transform the different components of the non-adapted UI model (non-adapted PIM) to adapted components based on the knowledge of the adapted modality provided by the former transformation. Thus, we have used component mapping rules. The component mapping rules match a component E of the non-adapted UI meta-model to its corresponding element E' of the adapted UI meta-model (PIM metamodel) using the already adapted modality Mod' . A component mapping rule is defined as a function (2) m_{comp} defined as follows:

$$\begin{aligned} m_{comp} : MM_{nonadapted} \times \{Mod'\} &\rightarrow MM_{adapted} \\ (E_{UI_{nonadapted}}, Mod') &\mapsto E'_{UI_{adapted}} \end{aligned} \quad (2)$$

Where:

$MM_{nonadapted}$ is the source non adapted meta-model

$\{Mod'\}$ is a set of the adapted modalities. Mod' is the adapted modality

$MM_{adapted}$ is the target adapted meta-model

Input: $Mod_{UI_{nonadapted}} \in MM_{nonadapted}, IOO$

Output: $Mod'_{UI_{adapted}} \in MM_{adapted}$

```

foreach modality  $Mod \in MM_{nonadapted}$  do
  switch  $IOO.DisabilityProfile$  do
    case Blindness do
      |  $return\ Mod' = Vocal$ 
    end
    case LowVision do
      |  $return\ Mod' = Multimodal$ 
    end
    case Deafness do
      |  $return\ Mod' = Graphic$ 
    end
    case HardDeafness do
      |  $return\ Mod' = Multimodal$ 
    end
  end
end

```

Algorithm 1 Modality mapping rule.

$E_{UI_{nonadapted}}$ is component of the UI non adapted model (non adapted PIM) conformed to the source meta-model $MM_{nonadapted}$

$E'_{UI_{adapted}}$ is the component of the adapted UI model (adapted PIM) conformed to the target meta-model $MM_{adapted}$

m_{comp} is implemented by the Algorithm 2. The complexity is also of the order of $O(n)$, where n is the number of classes of the meta model representing the non adapted UI

Input: $E_{UI_{nonadapted}} \in MM_{nonadapted}, Mod', IOO$

Output: $E'_{UI_{adapted}} \in MM_{adapted}$

```

foreach component  $E \in MM_{nonadapted}$  do
  switch  $Mod'$  do
    case (Vocal) do
      |  $return\ VocalComponent$ 
    end
    case (Graphical) do
      |  $return\ GraphicalComponent$ 
    end
    case (Multimodal) do
      |  $return\ MultimodalComponent$ 
    end
  end
end

```

Algorithm 2 Component mapping rule.

In order to generate an adapted PIM model, after applying the different mapping rules, we obtain adapted UI-components and not an adapted UI. So, we need some *Adaptation rules* to put the adapted UI components in a structure in order to build the adapted UI model. The adaptation process is based on these rules which are based on the mapping rules already defined. Thus, an adaptation rule specifies the container within which the adapted component should be contained. It is applied for each adapted component to build the adapted UI model. To define the adaptation rules, we have to specify a list of adaptation strategies based on the knowledge of the user disability profile. These strategies are essentials to restructure the already adapted components within the obtained UI model.

4.2 Refined transformation PIM2PIM (T2)

The refined transformation *PIM2PIM* generates a *refined adapted UI model* which depends on the adapted modality. It concerns the refinement and the extension of the structure of the adapted UI model based on the novel modality. As represented into Fig. 5, we transform the *Blindness-PIM meta-model* to *Vocal PIM meta-model*.

The refined PIM2PIM transformation is also based on a set of refined mapping and adaptation rules.

4.2.1 Refined component-mapping rules

The refined component-mapping rules refine each adapted component to a more concrete adapted component suitable to the adapted modality.

In the case of the refined component-mapping, a refined mapping rule matches an element E' of the adapted UI meta-model to its correspondent element E'' of the modality-based adapted UI meta-model using the adapted modality Mod' provided by the transformation $T1$.

A refined component mapping rule is defined as a function mm_{comp} . This function (3) is defined as follows. Function (3) is implemented by the Algorithm 3 with linear complexity (depending of the component type and number) from which the algorithm converges.

$$mm_{comp} : MM_{adapted} \times Mod' \times OO \rightarrow 2^{modMM_{adapted}} \\ (E'_{UI_{adapted}}, Mod', IOO) \mapsto E''_{UI_{adapted}} \quad (3)$$

Where:

- OO is the accessibility context ontology
- $MM_{adapted}$ is the source adapted meta-model
- $\{Mod'\}$ is the set of the adapted modalities. Mod' is the adapted modality
- $2^{modMM_{adapted}}$ is the set of parts of the target refined adapted meta-model dependent of the adapted modality
- $E'_{UI_{adapted}}$ is a component of the adapted UI model conformed to the source meta-model $MM_{adapted}$
- IOO is the instance of the accessibility context ontology specifying the user with disability profile.
- $\{E''_{UI_{adapted}}\}$ is the set of refined components of the component $E'_{UI_{adapted}}$ belonging to the refined adapted UI model (refined adapted PIM) conformed to the target meta-model $modMM_{adapted}$

where:

```

Input:  $E'_{UI_{adapted}} \in MM_{adapted}, Mod', IOO$ 
Output:  $E''_{UI_{adapted}} \in modMM_{adapted}$ 

foreach component  $E' \in MM_{adapted}$  do
  switch  $Mod'$  do
    case (Vocal) do
      | RefinementVocalCompMapping( $E'$ )
    end
    case (Graphical) do
      | RefinementGraphicalCompMapping( $E'$ )
    end
    case (Multimodal) do
      | RefinementMultimodalCompMapping( $E'$ )
    end
  end
end

```

Algorithm 3 Refinement mapping rules.

RefinedVocalCompMapping(E'), *RefinedGraphicalCompMapping*(E') and *RefinedMultimodalCompMapping*(E') are procedures getting as inputs adapted components and providing as outputs the correspondent refinement components in function of the given modality.

For example, the *RefinementVocalComponentMapping*(E') matches the *VocalInputInteractor* component to three Vocal Components as a user with blindness needs to interact with the system vocally using a *VocalPrompt* component to guide the user during the input operation while the *VocalInput* used to get the information from the user which is associated to a *Record* component to save the given vocal information (Cf. Fig. 6).

We illustrate this mapping through the example of the *Modality – GroupingUNIT*. In Fig. 7, we shows the different possible refinement of the component *GroupingUNIT* which depends on the already adapted modality *Modality – GroupingUNIT*. The latter can be transformed to *Modality*-{ Group, Form, Menu, Confirmation }. To select the suitable type of the refined component *GroupingUNIT*, we are based on the data involved into the attribute *Container Type*(Form, Menu, Panel,) of the class *GroupingUNIT* from the adapted UI model.

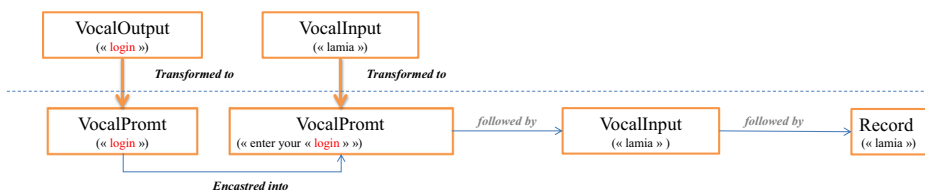


Fig. 6 Refined VocalInputInteractor per each adapted PresentationUnit

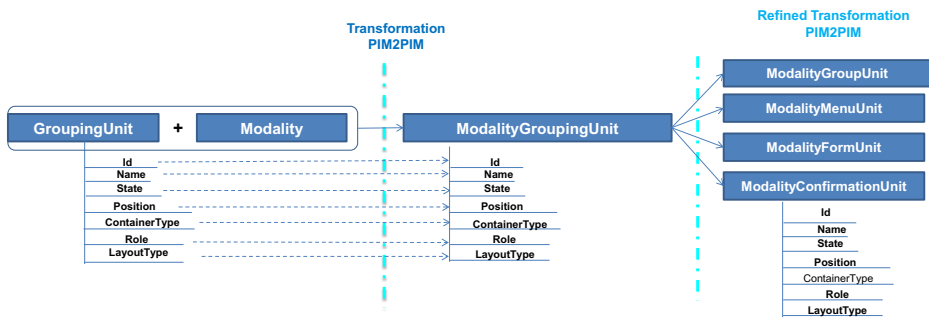


Fig. 7 Refinement adaptation of the GroupingUnit component

4.2.2 Refined component-adaptation rules

The adaptation of refinement component is applied to the case of blindness and low vision disability profiles (vocal and multi-modal modality). The refined component adaptation rules concern their restructuring in containers. So, each component of the refined adapted UI model has to be located within a novel structure depending on its type and the already adapted modality. The novel structure helps users with disabilities to achieve their tasks easily by adding new components if necessary. For instance, we have proposed to add a confirmation input procedure, regrouping a set of components in order to assist a user with blindness during the input operation with reporting to him/her the type of the previous action and to confirm the transition to the next action or to cancel it.

The example of Fig. 8 illustrates the process of the refinement of the component *VocalInput*. This component is placed within the vocal container *VocalGroupingUNIT* with its associated *VocalOutputInteractor*. All these components are placed within the component *VocalPresentationUNIT* which is presentation unit. In fact, the refinement of the component *VocalInputInteractor* corresponds to build a new structure has to be composed the following components:

- *VocalPresentationUNIT*: it represents a vocal presentation unit which includes the components *VocalGroup*, *VocalForm* and *VocalConfirmation*. The mapping

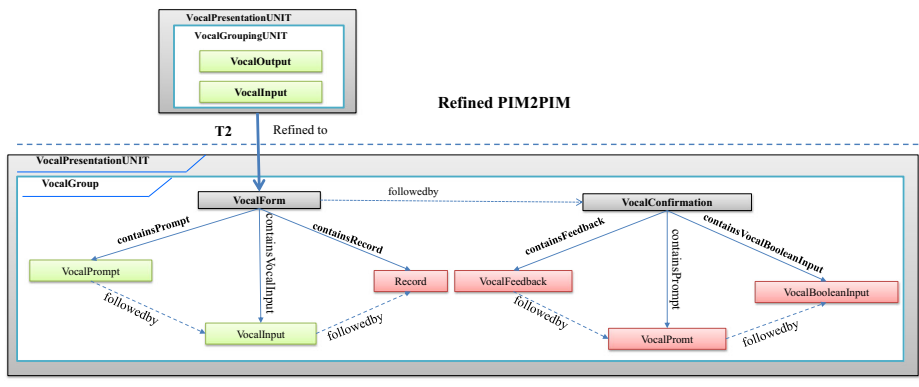


Fig. 8 Refinement adaptation of the VocalInput component in the case of vocal modality

(mm_{comp}) of the *VocalPresentationUNIT* provides a new *VocalPresentationUNIT* which differs from the initial one for the set of components and consequently for the structure. This mapping is specified as follows:

$$mm_{comp}(VocalPresentationUNIT, Vocal, Blindness) = \\ VocalPresentationUNIT$$

The *VocalPresentationUNIT* could be a *web page* when the specific platform is the *web*.

- *VocalGroup*: It represents the principal vocal container which must include the two vocal containers *VocalForm* and the *VocalConfirmation*. It is resulted from the mapping of the *VocalGroupingUNIT*:

$$mm_{comp}(VocalGroupingUNIT, Vocal, Blindness) = \\ VocalGroup$$

- *VocalForm*: It represents a vocal container having as type *form*. This container has to contain: a component *VocalPrompt* resulted from the transformation of the component *VocalOutput* which must be followed by the *vocalInput* and a component *record* to record and save the introduced vocal input

$$mm_{comp}(VocalInputInteractor, Vocal, Blindness) = VocalPrompt \\ \&VocalInput\&Record$$

- *VocalConfirmation*: It represents a vocal confirmation container. Its role is to confirm the introduced information by the component *VocalInputInteractor*. For better assistance of blind users, this container has to include these components:
 - A component *VocalFeedback* used to reply the content of the component *Record*. It must be followed by a component *VocalPrompt*
 - A component *VocalPrompt* used to ask a user with blindness if he/she confirms what was introduced via the component *VocalInputInteractor*. Each *VocalPrompt* has standard expressions to formulate a question or an answer. The construction of these expression satisfies a given grammar.
 - A component *VocalBooleanInput* used to report the answer to the question. The content used to confirm the user's input must be expressed simply by *Yes/No*.

4.3 Specific transformation PIM2PSM (T3)

T3 concerns the transformation of the already *refined adapted UI model* to a Platform Specific Model (PSM). It allows the adaptation of the *adaptedUI-PIM* to an execution platform.

As its represented in Fig. 5, we have $3^{PlatformNumber}$ specific transformations as for each execution platform we can transform the UI using one of the 3 modalities (vocal, graphical, multimodal). In our work, we consider only three execution platforms: *mobile platform*, *web platform* and *desktop platform*.

Based on the kind of the execution platform, we generate a specific model (PSM) dependent of the platform such as *VocalMobile-PSM*, *VocalDesktop-PSM*, *MultimodalWeb -PSM* (Cf. Fig. 5). Hence, T3 takes as input the *refined adaptedUI-model (PIM)*, the type of platform and the accessibility context ontology instance.

The goal of T3 is to adapt the style (color, font-family, background-color, text-align, font-size, position, layout, width, height, volume-level, phone-icon, microphone-icon, etc) of the adapted and refined structure to the specific platform considering different contextual parameters for the adaptation such as the technical context, the environment context, the user preferences and the user situation.

4.3.1 Specific mapping rules

The PIM2PSM transformation is based on specific mapping rules to match the specification of each target platform. We have respected the definition of the accessibility by following the recommendation proposed by the norms *WCAG2.0* and the *ISO 9241-171* using their directives.

A specific mapping rule transforms only the style of the *adapted UI-component* resulted from the previous transformation T2. The component-style is defined by different properties like color, font-size, width, height, layout, relative and absolute position, alignment, volume-level, phone-icon, microphone-icon, contrast defined by the background color and the font-color, etc. The adapted-PSM follows and fits different contextual parameters such as the device type, the noise level, the luminosity level, the user identity and his/her preferences and the user situation describing the mobility. These parameters are depicted from the ontology. To obtain them, we use an ontology reasoner based on inference rules specified in an SWRL [31] file. The use of this reasoner is imperative as the number of contextual parameters is quite important.

A specific mapping rule matches a refined component to its correspondent specific component E'' based on the already adapted modality Mod' and considering several contextual parameters. The style of each component E'' is designed through attribute-styles according to the modality value:

- *graphical modality*: For graphical platforms (web, desktop or mobile), we consider attributes such as the width, the height, the text-size, the text-color, the text-font, the background-color, the contrast. These attributes depend on the size of the device screen size.
- *vocal modality*: For vocal platforms, we consider attributes such as the volume level, phone-icon, microphone-icon, the grammar expression

In the case of web platforms, the style is described by a *css file* attached to the web page. Furthermore, the integration and the use of the technology of Responsive Web design (RWD) [26] enhances the built of adaptable style files dependent on the device-type and its screen-size (large or small).

Thus, we have defined a function (4) mm_{scomp} specified as follows:

$$mm_{scomp} : modMM_{adapted} \times \{Platform\} \times OO \rightarrow PlatformMM_{adapted}$$

$$(E''_{UI_{adapted}}, Platform, IOO) \mapsto E''^{UI_{adapted}}(attr_1 = value, \dots, attr_n = value) \quad (4)$$

Where:

- $modMM_{adapted}$ is the adapted meta-model based on the adapted modality
- $\{Platform\}$ is the set of the target platforms (web, desktop, mobile) OO is the accessibility context ontology
- $PlatformMM_{adapted}$ is the adapted meta-model of the target platform

- $E''_{UI_{adapted}}$ is a non adapted element from the adapted meta-model based on the adapted modality $modMM_{adapted}$
- IOO is the instance of the accessibility context ontology where we consider the device type, the noise level, the luminosity level, the user identity and his/her preferences and the user situation describing the mobility.
- $E''_{UI_{adapted}}$ is the adapted element from the adapted meta-model to the target platform $PlatformMM_{adapted}$
- $attr_1..attr_n$ define the style attributes relative to the already adapted modality

For instance, for visually impaired users and with large screen size superior to 10" (laptop, desktop), we have considered that:

- the font-size of all the adapted textual component has to be equal to 26
- the contrast must be high which is defined by a foreground color black and a font-color white.

This mapping is formalised as follows:

$$mm_{scomp}(TextInput, web, Device - type = "desktop") \mapsto$$

$$input(type = "text", style = "font - size : 26,$$

$$foreground - color : black, font - color : white"$$

4.3.2 SWRL rules

We have introduced Semantic Web Rule Language (SWRL) [31] rules into the specific transformation to the execution platform in order to facilitate the reasoning on accessibility context. In fact, reasoning facilitates and affects the different forms of interaction with users with disabilities.

SWRL rules serve to implement adaptation strategies of the specific transformation to the execution platform. We reason on the accessibility context knowledge to allow the adaptation according to the platform and other contextual parameters such as the technical context (device-type, device screen-size, input and output devices), the user identity and preferences, the physical and social environment conditions (time, location, proximity, noise and light level) and the user situation (user mobility). The following rule 1 specifies in case of PC Device Type and in the case of Low Vision Disability, we deduce that the text size should be 24 with a zoom = 100% and with high volume.

```

AccessibilityContext(?x)^hasTechnicalContext
(?x,?y)^DeviceType(?y,"PC")^hasUserIdentity
(?x,?u)^hasPreferences(?u,?p)^hasDisability
Profile(?x, ?dp)^isASensory(?dp,?s)^isALowVision
(?s,?l) -> TextSize(?p,24)^Zoom(?p,100)^Volume
(?p, "High")

```

[1]

The following rule 2 is applied when user disability profiles are low vision and the platform is a PC. It selects the interaction devices of the platform. So, only the keyboard and the

microphone are used as input interaction devices and the screen and loudspeaker as output interaction devices.

```
AccessibilityContext(?x)^hasDisability
Profile(?x,?y)^isALowVision(?y,?p)^
DeviceType(PC)-> InputDevices(Microphone)^      [2]
InputDevices(Keyboard)^ OutputDevices(Screen)
^OutputDevices(LoudSpeaker)
```

The rule 3 personalizes preferences for visually impaired users with a low severity by maintaining volume to medium and the text size to 16 and the zoom to 50%.

```
AccessibilityContext(?x)^hasTechnical
Context(?x,?y)^hasUserIdentity(?x,?u)^
hasPreferences(?u,?p)^hasDisabilityProfile
(?x, ?dp)^correspondstoUser(?dp,?u)^
isALowVision(?dp, ?l)^Severity(?l,"Low")          [3]
->TextSize(?p, "16"^^xsd:int)^Zoom
(?p, "50"^^xsd:int)^Volume(?p, "Medium")
```

In the case of blind people, we apply the following rule 4 to increase the volume of the platform system:

```
AccessibilityContext(?x) ^ hasDisability
Profile(?x, ?dp) ^ isABlindness(?dp, ?b) ^
hasUserIdentity(?x, ?u) ^
hasPreferences(?u, ?p) ^ correspondstoUser      [4]
(?dp, ?u) -> Volume(?p, "High")
```

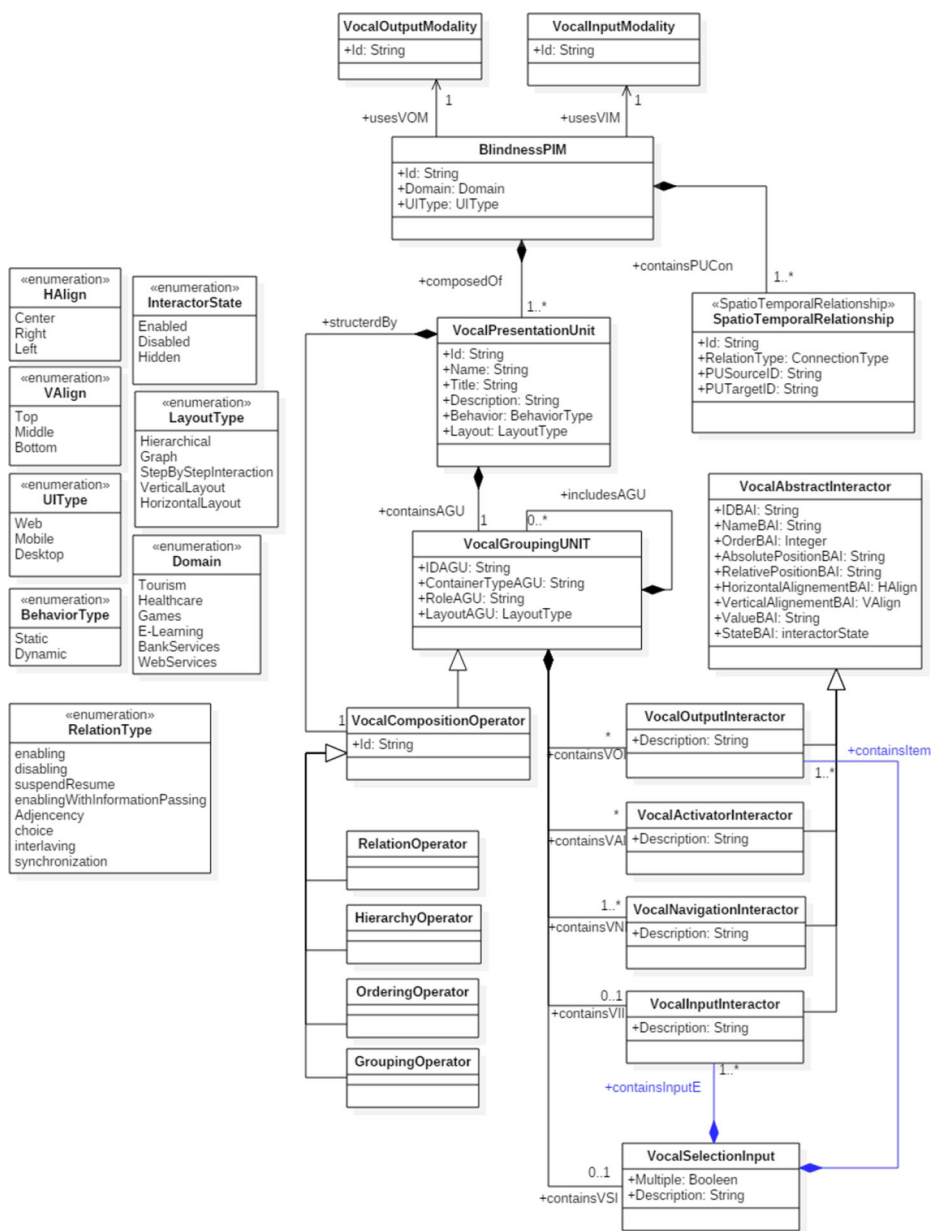
The rule 5 is applied in the case of blind people user. When he/she uses a device-type “desktop/laptop/tablet”, only the microphone is used as an input interaction device and the loudspeaker as output interaction device.

```
AccessibilityContext(?x) ^ hasDisability
Profile(?x, ?dp)^ isABlindness(?dp, ?b) ^
hasTechnicalContext(?x, ?y) ^
hasInteractionCapacities(?y, ?i) ^              [5]
isAInput(?i, ?in) ^ isAOutput(?i, ?out) ^
Model(?y, "PC") -> Microphone(?in) ^LoudSpeaker(?out)
```

4.4 Case study for blindness disability

Blindness disability profile is defined as a sensory disability which necessitate a selection of adaptation strategies that bridge the visual disability. So, four strategies have been put in place to specify transformation rules [39] and used to build the Blindness-PIM metamodel specific to blindness (Fig. 9):

- **Action 1:** Adapts graphical components to vocal components by applying mapping rules defined by the Algorithm 1 followed by the Algorithm 2. An example of application of the mapping rules dedicated to the blindness handicap, in the case of the Input Interactor after having adapted the input and output modalities. The R_{B5}



permits to generate an adapted component of type *VocalInput* from any type of the element *InputInteractor*.

$$m_{mod}(InputModality, Blindness) = VocalModality$$

$$m_{mod}(OutputModality, Blindness) = VocalModality$$

$$R_{B5} : m_{comp}(InputInteractor, Blindness) = VocalInput$$

- **Action 2:** Action 1 concerns only adapting UI-components to the vocal modality. Unfortunately, this does not solve the problem and requires another actions. In fact, the use of voice interaction is not as easy as it seems for the blind. The UI needs to be further simplified by minimizing the number of components per Presentation Unit. This issue appears when the user is faced with filling in a form. To this end, we will adopt an approach aiming at minimizing the number of input elements of type *InputInteractor* into the *Grouping Unit*. Therefore, we recommend to have only one *InputInteractor* per grouping unit of type *GroupingUnit*. Each *PresentationUnit* which consists of all the perceivable parts of the UI and encloses all the abstract components of the UI, should only contain one *GroupingUnit*. This leads to new generated Presentation Units, each of them contains a unique component *InputInteractor*. This will rely on the adaptation rules according to developed custom algorithms.
- **Action 3:** By continuing the *Action 2*, we results a number of adapted *PresentationUNITs* generated from the adapted UI and linked by some supplementary buttons. This requires a strategy to ensure the dialogue between these different *PresentationUNITs*. For that reason, we append some extra navigational items: *NavigationInteractor* labeled *next* and *previous* enabling the navigation between the respective pieces of *PresentationUNIT*.
- **Action 4:** After having obtained a series of Presentation Units each containing a component labelled *InputInteractor*, we proceed to the refinement of the already adapted vocal components. The refinement requires specific refinement functions for each component kind. The example shown on the Fig. 6 illustrates a refinement of the voice input component by introducing additional elements necessary for voice interaction.

The goal of this case study is to assess the suitability of the adaptation process for the metamodel transformations that are implemented. In this case study, we only concentrate on the adaptation process of the hotel booking UI for blind users using the mobile computer. Figure 10 depicts the hotel booking UI model and its associated PIM model represented by the Ecore model [6] to describe the UI model. To start, we need to determine the type of non-adapted UI, its application domain, and the input and output interaction modalities.

- **Type:** The Hotel Booking UI is of type *Desktop*. It executes on a desktop or laptop computer.
- **Domain:** This UI is assigned to the tourism field.
- **InputModality:** the UI employs the graphical modality as input because the interaction with this GUI is done graphically in order to enter information through the form.
- **OutputModality:** the UI employs the graphical mode to displays the data through graphical elements of the interface.

As represented in the Fig. 10, the non adapted UI model has a single element *PresentationUnit* labeled *HotelReservation* which is built upon a number of components of types *GroupingUnit*, *InputInteractor*, *OutputInteractor* and *ActivatorInteractor*. The repartition of the elements within each *Grouping Unit* is as follows:

- The component *GroupingUnit* labelled *Formulaire* includes three *TextInput* components and their associated *TextOutput* components: *TextInput*("Name"), *Textoutput*("Name"), *TextInput*("Telephone"), *Textoutput*("Telephone"), *TextInput*("Email"), *Textoutput*("Email")

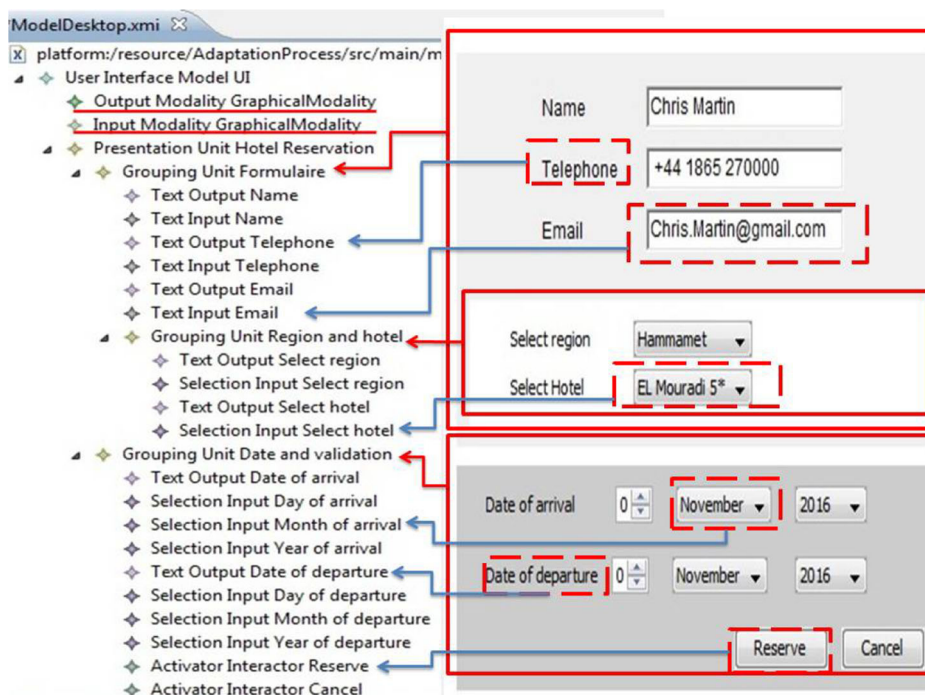


Fig. 10 Hotel Booking Non adapted PIM Model by Ecore

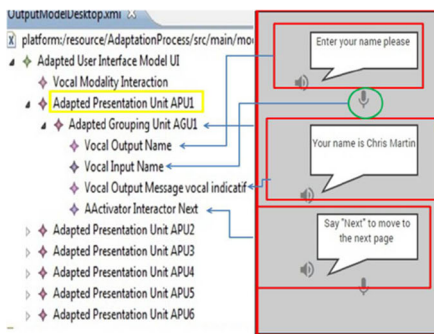
- The GroupingUnit called *Region and hotel* has two SelectionInput components with their labels Textoutput: {TextOutput(“SelectHotel”), SelectionInput(“Selecthotel”)} and {TextOutput(“Selectregion”), SelectionInput(“Selectregion”)}
- The GroupingUnit labelled *Date and validation* comprises three SelectionInput components with their associated labels TextOutput. It also has two activator components: *Reserve* and *Cancel*.

Based on the PIM meta-model suited to blindness, and following the four actions, the resulting vocal UI is a combination of the six resulting UIs as shown in Fig. 11(a), (b), (c), (d), (e) and (f) where in each UI we .

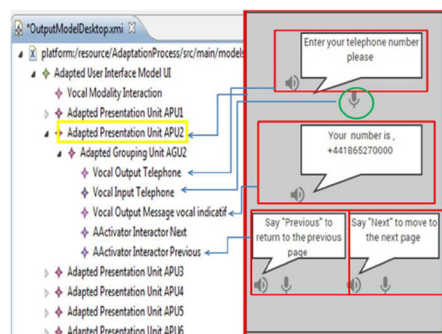
5 Empirical study

As an important quality metric [30], usability is described by ISO 9241-11 [13] as the measure to which a product can be used to complete specified goals with effectiveness, efficiency and satisfaction in a specified context of use by specific users. The usability of AUIAC enables us to assess and measure the extent to which the highlighted strategy based on the adaptation of modalities could provide useful and accessible UIs.

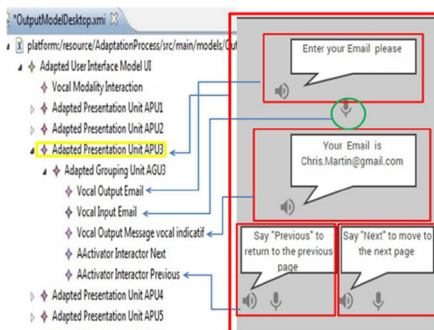
Consequently, we considered a number of usability factors in our empirical study, such as:



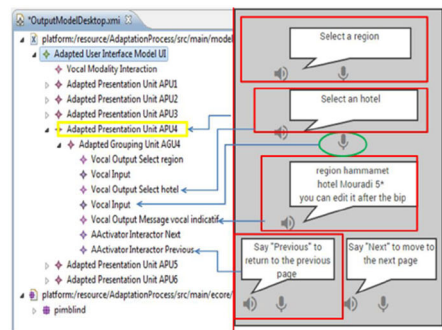
(a) PresentationUNIT 1: UI1



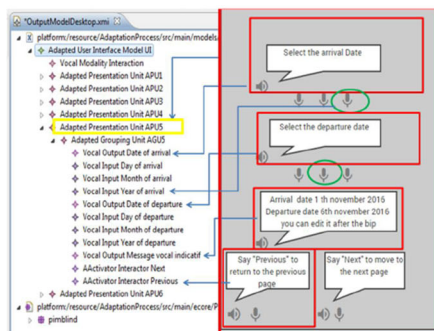
(b) PresentationUNIT 2: UI2



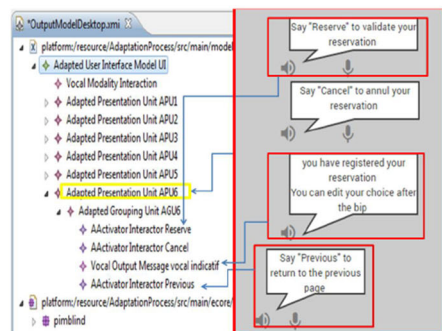
(c) PresentationUNIT 3: UI3



(d) PresentationUNIT 4: UI4



(e) PresentationUNIT 5: UI5



(f) PresentationUNIT 6: UI6

Fig. 11 Adaptation of the Booking Hotel UI based on the metamodel transformation for blind users, which has produced more than six new generated and adapted UIs

- *The effectiveness:* It describes the accuracy and the completeness with which users achieve specified goals of the system. The effectiveness of a the AUIAC system was evaluated by measuring some metrics such as:

- **Completion Rate:** It measures whether users succeed in achieving their goals when working with a system. It is calculated by assigning a binary value of ‘1’ if the test participant manages to complete a task and ‘0’ if not.
- **Number of Errors:** It measures the number of errors committed at each completion task. The error is equal to 100% when the completion rate is 0. According to the employed modality, we have defined different kind of errors. In the case of the vocal modality, we have defined errors related to: Command-pronunciation, matching-grammar prediction, input-error, irrelevant actions (mistake, omission).
- *The efficiency:* It describe how much capability is enabled to end user to expend appropriate amounts of resources for the effectiveness achievement. In our case, we need to measure the task completion time [27].
- *The satisfaction:* It defines user-acceptance towards the product. To do that, we are based on the System Usability Scale (SUS) questionnaire.
- *Learnability:* It defines how much it is easy to quickly learn system functionalities.

In the Table 1, we provide a description of participants’ classification. We have 90 subjects classified into three groups of people with disabilities (*Class-Blind*, *Class-Low Vision*, *Class-Deaf*) based on their experience with digital devices (low, medium, high). In addition, we have developed two appropriate applications for participants to be carried by them. For Blind and Visually Impaired users, we have provided a hotel reservation form to be filled out and a Video Player Interface for Deaf users. The established test is just a step for further prototyping, user testing and design refinement. In order to measure learnability and satisfaction rate, we have relied on a pre-designed questionnaire named the System Usability Scale (SUS) [18] since it is a reliable and low-cost usability scale employed for general system usability evaluation. It consists of 10 items. Basically, usability is expressed by eight items: *Q1*, *Q2*, *Q3*, *Q5*, *Q6*, *Q7*, *Q8* and *Q9*. As for the learnability, it is described by two items: *Q4* and *Q10*. Both usability and learnability scales are considered reasonably reliable with an alpha coefficient of 0.91 and 0.70 respectively. Though the scores ranges from 0 to 100, these are not percentages and should be considered only in terms of their percentile ranking. In order to measure User Satisfaction, users with disabilities have to fill the questionnaire after conducting the test. They are asked to provide a 5 – *point* scale numbered from 1 anchored with “Strongly disagree” to 5 anchored with “Strongly agree”. Besides, subjects are recommended to follow a unified scenario:

- Be aware of the study’s objectives, be confident in the results, and make sure there is no denigration or minimization of their skills.
- Gathering personal data about each participant in order to build a dataset and to populate our Knowledge Base. Data is about name, gender, date of birth, culture, language skills,

Table 1 Participants classification

	Class-Blind	Class-LowVision	Class-Deaf	Total
Low UX-friendly	12	7	5	24
Medium UX-friendly	15	18	10	43
High UX-friendly	3	4	16	23
Total	30	29	31	

Table 2 Task completion results

Task completion (%)	Class-Blind			Class-LowVision			Class-Deaf		
	L	M	H	L	M	H	L	M	H
	10	31.02	75.83	31.83	66.14	89.36	10.20	61.11	91.69

educational level, device-control level, a score towards the need for an IT solution, device-use frequency in term of number of hour per day, etc.

- Participating to a training session to master the manipulation of user interface prototypes prepared to explain how to accomplish tasks using the appropriate modalities. The importance of this step in making it easier for people with disabilities to accomplish tasks is evident.
- Assistance on demand for helping them only if the participant finds difficulty to do a task.

To make the test results meaningful, we dispose of metrics to evaluate the appropriateness of the modality-based adaptation process in terms of task completion rate, the error rate, user satisfaction and learnability.

5.1 Results

Results are generated by the statistical tools SPSS [12] and STATA [33].

In Table 2, we means the task achievement percentage *TaskCompPer* of the two interfaces regrouped for each disability class based on the UX-friendly level (Low, Medium, High). In fact, the TaskCompPer varies from 10% to 75.83% for blindness disability from low to high UX-friendly. For low vision class, it goes from 31.83% to 89.36%. Finally, for deaf disability, the TaskCompPer varies from 10.20% to 91.69% according to UX-friendly level. Task completion rate is different for deaf PWDs because listening is the main barrier for deafness. The results show an acceptable rate in exception to PWDs with low experience with computing devices. We find that task completion is more efficient with a good experience with the computer. This conclusion leads us to focus on enhancing the experience of users with disabilities to make better use of UIs on interactive systems specially blind and low vision, to better interact on interactive systems using vocal or multi-modal modalities.

In addition, Table 3 exposes results of error rate from the 90 observations. Indeed, we have obtained an average of error rate ranging between 35.6% to 89.5% for Class-Blind and between 19.02% to 63.44% for Class-LowVision from High UX-friendly to low UX-friendly. These values confirm the two already mentioned considerations that assume that despite modality's effectiveness as a means of interaction, user friendliness remains an important factor explaining some failures. In the other side, deaf PWDs are in a comfortable state with an average rate between 75.46% to 11.20%, because and for the same reason, the graphical modality provides the most effective manner of interacting with digital devices

Table 3 Error rate results

Error Rate (%)	Class-Blind			Class-LowVision			Class-Deaf		
	L	M	H	L	M	H	L	M	H
	89.5	66.6	35.6	63.44	24.82	19.02	75.46	29.19	11.20

Table 4 User satisfaction rating

	Min	Max	Mean	Std.dev
Satisfaction rate	2	7	5	1.45

where the only impediment for deaf people is when video or audio is involved in the user interface.

User satisfaction measures the behavior and the attitude of users towards the proposed system. In our case and at the end of the test, we have asked the 90 participants to give a rank from 0 to 7 expressing their level of satisfaction towards the adaptation strategies. The Table 4 shows the calculated average of user satisfaction where the mean value is equal to 5. Based on this value, we find that the solution as a whole is considered acceptable and satisfactory.

Table 5 shows SUS scores by UX-friendly and by disability classes. The results summarized in the Table 5 reveal that the SUS score is = 74.36. The internal consistency of the responses achieved a Cronbach's Alpha of 0.774 which is a good score for SUS which received an *B* grade according to [28].

While the value of Cronbach's alpha is 0.774 so this questionnaire included into the level of reliable as represented in Table 6. In addition, regarding the usability and learnability factors, it is found that the reliability of usability is reasonable and acceptable with a $\alpha = 0.73$, however, the learnability scale is considered questionable with an $\alpha = 0.49$ which is considered quite reliable. Besides, Table 6 focuses on the correlation of each factor with the SUS's overall. The usability factor is more correlated with the SUS's overall with a value of 0.965 whereas the learnability factor is less correlated with the SUS and has a value of = 0.76. The results of the questionnaire experiment found clear support for the ease of use in terms of usability whereas there are problem with learnability which raise novel challenges.

5.2 Discussion

Based on the results obtained regarding usability metrics related to effectiveness, efficiency, satisfaction and learnability, we find that the overall results are good and verify their acceptability. Consequently, we consider that adopting modality-based interaction strategies to adapt the UI is useful and feasible. These strategies support the user with disability's interaction with any adaptive and multimodal interactive system.

Table 5 SUS scores by UX-Friendly and by disability

	N	Mean	std.dev	Min	Max
Low UX-Friendly	24	72.91	7.16	60	87.5
Medium UX-Friendly	43	72.85	9.86	52.5	82.5
High UX-Friendly	23	90	11.51	60	100
Average		74.36	13.85	40	100
Blind	30	85.8	6.02	82.5	97.5
Low vision	29	72.91	7.16	60	87.5
Deaf	31	67.4	16.9	40	100
Average		74.36	13.85	40	100

Table 6 Reliability statistics

	N f items	Alpha's Cronbach	SUS.Score	Corr.SUS	Std.dev
Usability	8	0.73	59.25	0.9654	10.83
Learnability	2	0.49	15.11	0.7686	4.41
Total	10	0.774	74.36		

The Table 6 reveals from all users with the different UX-friendly finds it usable to interact with the system but it is difficult to learn rapidly on how do interact with the UIs by using the mentioned interaction modalities. This indicates that users with disabilities found the system user-friendly and usable but it was challenging for them to learn to manipulate the system. This may be due to the relationship of this class of people to technology and familiarity with computing platforms. It is worth pointing out that the problem stands out for blind and visually impaired people. In contrast, the deaf people encounter less problems when they interact with graphical modality. Significantly, these findings require further improvement when using the vocal modality. The enhancement is decided based on the obtained values in relation to the task completion and the error rate. In fact, vocal interaction, in distinction to classical GUI interaction, is an emerging mode of interaction on smart devices that has shown some inconvenience and unpleasantness to some ordinary and able-bodied users. It appears natural for blind people to experience discomfort when interacting with this type of modality. At the same time, when considering the UX-friendly, very experienced PWDs have shown a great ability of learning and satisfaction towards system.

Modality-based adaptation strategies are very important and have the potential to provide a significant solution for people with disabilities to interface with any kind of digital device. However this is not the only way to go. Actually, we relied on the WCAG2.0 guidelines [11] to build the AUIAC system and we have integrated Artificial Intelligence in several AUIAC system levels [38] to overpower and improve some manual process.

6 Conclusions

In this paper, we have introduced our AUIAC framework for the adaptation of UI to the accessibility requirements. The AUIAC system is made up of four modules: the accessibility knowledge management module (AccKM module), the user interface reverse engineering module (UIRE module), the accessibility adaptation process module (AccAP module) and the cross-platform adaptation process module (CrossPAP module).

We conducted some experiments based on usability evaluation to prove the reliability of the suggested adaptation process. The empirical study that we conducted gave acceptable outcomes. It has demonstrated that employing multimodal interaction enhances the efficiency and the satisfaction of the people with disabilities. For this reason, we consider that the use of modalities should be a core requirement for accommodating PWDs followed by more improvements.

As further work, we work toward improving the interaction between people with disabilities and the user interface by incorporating the artificial intelligence, with a particular regard to the Reinforcement Learning [38].

Data Availability The datasets used during the current study are available from the corresponding author on reasonable request.

Declarations

Conflict of Interests On behalf of all authors, the corresponding author states that there is no conflict of interest.

References

1. Abascal J, Aizpurua A, Cearreta I, Gamecho B, Garay N, Miñón R (2011) A modular approach to user interface adaptation for people with disabilities in ubiquitous environments. In: Internal technical report No. EHU-KAT-IK-01-11
2. Abrahão S, Insfran E, Sluÿters A, Vanderdonckt J (2021) Model-based intelligent user interface adaptation: challenges and future directions. *Softw Syst Model* 20(5):1335–1349
3. Akiki PA, Bandara AK, Yu Y (2016) Engineering adaptive model-driven user interfaces. *IEEE Trans Softw Eng* 42(12):1118–1147. <https://doi.org/10.1109/TSE.2016.2553035>
4. Bendaly Hlaoui Y, Zouhaier L, Ben Ayed L (2018) Model driven approach for adapting user interfaces to the context of accessibility: case of visually impaired users. *J Multimodal User Interfaces*
5. Burzagli L, Emiliani PL, Antona M et al (2021) Intelligent environments for all: a path towards technology-enhanced human well-being. Universal Access Information Society
6. EcoreTools (2017) Graphical Modeling for Ecore. <http://www.eclipse.org/ecoretools/>. Accessed Jun 2021
7. Engel C, Müller EF, Weber G (2019) SVGPLott: an accessible tool to generate highly adaptable, accessible audio-tactile charts for and from blind and visually impaired people. In: Proceedings of the 12th ACM international conference on pervasive technologies related to assistive environments. ACM, pp 186–195
8. Firmenich S, Garrido A, Paternò F, Rossi G (2019) User interface adaptation for accessibility. In: Yesilada Y, Harper S (eds) Web accessibility. Human–computer interaction series. Springer, London
9. Gajos Z, Weld DS, Wobbrock JO (2010) Automatically generating personalized user interfaces with Supple. *Artif Intell* 174:910–950
10. Gonçalves TG, Oliveira KM, Kolski C (2018) Identifying HCI approaches to support CMMI-DEV for interactive system development. *Comput Stan Interfaces* 58:53–86
11. Henry SH (2010) Web content accessibility guidelines (WCAG) 2.0. <https://www.w3.org/WAI/GL/WCAG20/>. Accessed 2017
12. IBM (2019) SPSS software, Ibm.com. <https://www.ibm.com/analytics/spss-statistics-software>. Accessed 24 Feb 2020
13. ISO (1998) Ergonomics Requirements for office work with visual display terminals (VDTs)-Part 11: Guidance on Usability
14. ISO (2008) I S O 9241-171, Ergonomics Of human-system interaction Part 171: Guidance on software accessibility
15. ISO (2017) ISO/IEC DIS 40500 Information technology, W3C Web Content Accessibility Guidelines (WCAG) 2.0 (TC/SC: JTC 1)
16. Khan A, Khusro S (2019) Blind-friendly user interfaces a pilot study on improving the accessibility of touchscreen interfaces. *Multimed Tools Appl* 78:17495–17519
17. Kosmas P, Galanakis G, Constantinou V, Drossis G, Christofi M, Klironomos I, Zaphiris P, Antona M, Stephanidis C (2020) Enhancing accessibility in cultural heritage environments: considerations for social computing. *Univ Access Inf Soc* 19:471–482
18. Lewis JR, Sauro J (2009) The factor structure of the system usability scale. In: Human centered design. Volume 5619 of the series Lecture Notes in Computer Science, Springer, pp 94–103
19. Mitrovic N, Bobed C, Mena E (2016) A review of user interface description languages for mobile applications. In: the Tenth international conference on mobile ubiquitous computing, systems, services and technologies UBICOMM
20. Muller PA, Fleurey F, Jezequel JM (2005) Weaving executability into object oriented meta-languages. *Model Driven Engineering Languages and Systems*, pp 264–278
21. OMG (2001) Model Driven Architecture (MDA) document number ormsc/2001-07-01
22. Oliveira RF, da Mota Moura AM, Leite JCSP (2018) Reengineering for accessibility: a strategy based on software awareness. In: Proceedings of the 17th Brazilian symposium on software quality. pp 180–189

23. Paterno F, Santoro C, Spano LD (2011) A design space for user interface composition Model-Driven development of advanced user interfaces
24. Peissner M, Janssen D, Sellner T (2012) MyUI Individualization Patterns for Accessible and Adaptive User Interfaces. In: The First International Conference on Smart Systems, Devices and Technologies, pp 25–30
25. Peissner M, Schuller A, Ziegler D, Knecht C, Zimmermann G (2014) Requirements for the successful market Adoption of Adaptive user Interfaces for Accessibility. In: International conference on universal access in human-computer interaction, Springer, pp 431–442
26. Responsive Design (2020) Responsive web design basics. <https://web.dev/responsive-web-design-basics/>. Accessed 01 Jun 2020
27. Sauro J (2011) MeasuringU: what is a good task-completion rate? Measuringu.com. <https://measuringu.com/task-completion/> (accessed 2017)
28. Sauro J (2018) Interpreting single items from the SUS. Measuringu :11. <https://measuringu.com/sus-items/>
29. Scott MJ, Spyridonis F, Ghinea G (2015) Designing for designers: towards the development of accessible ICT products and services using the VERITAS Framework. *Comput. Stand. Interfaces* 42:113–124
30. Seffah A, Donyaee M, Kline RB, Padua HK (2006) Usability measurement and metrics: a consolidated model. *Softw Qual J* 14(2):159–178
31. Skillen KL, et al. (2014) Ontological user modelling and semantic rule-based reasoning for personalisation of help-on-demand services in pervasive environments. *Future Gener Comput Syst* 34:97–109
32. Souidi E, Zouhaier L, Hlaoui YB (2022) Iot for real-time accessibility ontology population to context-awareness adapt user interfaces. In: 2022 IEEE 46th annual computers, software, and applications conference (COMPSAC), pp 1462–1467. <https://doi.org/10.1109/COMPSAC54236.2022.00232>
33. Stata Statistical software for data science. <http://www.stata.com/>. Accessed 01 Jun 2020
34. Stephanidis C, Paramythis A, Sfyrakis M, Stergiou A, Maou N, Leventis A, Paparoulis G, Karagiannidis C (1998) Adaptable and adaptive user interfaces for disabled users in the AVANTI project. Springer, Heidelberg, pp 153–166
35. Vanderheiden GC, Treviranus J, Ortega-Moral M, Peissner M, de Lera E (2004) Creating a global public inclusive infrastructure (GPII). Springer, Cham, pp 506–515
36. WHO International classification of functioning, disability and health (ICF). <http://www.who.int>. Accessed 01 Apr 2017
37. Zouhaier L, Bendaly Hlaoui Y, Ben Ayed L (2017) Users Interfaces Adaptation for Visually Impaired Users Based on Meta-model Transformation. In: IEEE 41st annual computer software and applications conference
38. Zouhaier L, Hlaoui Y, Ayed L (2021) A Reinforcement Learning Based Approach of Context-driven Adaptive User Interfaces. In: 2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC), Madrid, Spain, pp 1463–1468. <https://doi.org/10.1109/COMPSAC51774.2021.00217>
39. Zouhaier L, Hlaoui Y, Ben Ayed L (2017) Methodology for the development of accessible user interfaces based on Meta-Model transformations bendaly the case of blind users. In: CISIM 2017, LNCS 10244, pp 73–84

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.